

ui.card 全面详细解析

ui.card 是 NiceGUI 框架中的核心容器组件，基于 Quasar 的 QCard 组件实现，提供带有阴影效果的容器功能，适用于组织和展示相关联的界面元素（如图片、文本、表格等），支持灵活的样式定制、布局调整和交互绑定，是构建结构化界面的关键组件之一。

一、核心特性与基础说明

1. 本质与设计初衷

- 基础功能：提供可视化容器，通过阴影效果区分界面层级，默认包含内边距以保证内容与容器边界的合理间距。
- 版本差异：2.0.0 版本后不再隐藏嵌套元素的外边框和阴影；若需兼容旧版 QCard 行为（无内边距 + 隐藏嵌套元素外边框 / 阴影），可使用 `tight()` 方法。
- 核心依赖：基于 Quasar 的 QCard 组件，支持继承 Quasar 的 props 和样式类，同时扩展了 NiceGUI 特有的绑定、资源管理等功能。

2. 初始化参数

参数名	类型	说明	默认值
<code>align_items</code>	str (可选 值: "start"、"end"、"center"、"baseline"、"stretch")	卡片内元素的对齐方式	None

二、常用功能与示例

1. 基础卡片（默认样式）

默认包含阴影和内边距，可直接嵌套各类 UI 元素（如标签、图片、表格等）：

```
from nicegui import ui

with ui.card():
    ui.label("基础卡片示例")
    ui.image("https://picsum.photos/id/684/640/360") # 嵌套图片元素

ui.run()
```

2. 无阴影卡片

通过两种方式实现无阴影效果，适用于扁平化设计场景：

方式 1: 使用 `no-shadow` 样式类 + 自定义边框

```
from nicegui import ui

with ui.card().classes("no-shadow border border-gray-200"): # 移除阴影+添加1px灰色边框
    ui.label("See, no shadow!")

ui.run()
```

方式 2: 使用 Quasar 的 `flat` 和 `bordered` 属性

```
from nicegui import ui

with ui.card().props("flat bordered"): # flat 移除阴影, bordered 添加边框
    ui.label("Also no shadow!")

ui.run()
```

3. 紧凑布局卡片 (tight 模式)

通过 `tight()` 方法移除内边距和嵌套元素间距, 同时隐藏嵌套元素的外边框 / 阴影, 还原原生 QCard 行为:

```
from nicegui import ui

rows = [{"age": "16"}, {"age": "18"}, {"age": "21"}]

with ui.row(): # 横向排列两个卡片对比
    # 普通卡片 (带内边距)
    with ui.card():
        ui.table(rows=rows).props("flat bordered")
    # 紧凑布局卡片 (无内边距)
    with ui.card().tight():
        ui.table(rows=rows).props("flat bordered")

ui.run()
```

4. 带分区的卡片

结合 `ui.card_section()` 实现卡片内容分区, 使结构更清晰:

```
from nicegui import ui

with ui.card().tight():
    ui.image("https://picsum.photos/id/684/640/360") # 顶部图片区域
    with ui.card_section(): # 文本分区
        ui.label("Lorem ipsum dolor sit amet, consectetur adipiscing elit...")

ui.run()
```

三、核心属性 (Properties)

属性名	类型	说明	
classes	Classes[Self]	元素的 CSS 类 (支持 Tailwind、Quasar 类)	
client	Client	该元素所属的客户端实例	
html_id	str	HTML DOM 中的元素 ID (2.16.0 版本新增)	
is_deleted	bool	元素是否已被删除	
is_ignoring_events	bool	元素是否正在忽略事件	
parent_slot	`Slot`	None`	元素的父插槽 (可设置)
props	Props[Self]	元素的属性 (支持 Quasar 组件的 props)	
style	Style[Self]	元素的内联 CSS 样式	
visible	BindableProperty	元素的可见性 (支持数据绑定)	

四、关键方法 (Methods)

1. 布局与样式相关

方法名	语法	说明
tight()	tight() -> Self	移除内边距和嵌套元素间距, 隐藏嵌套元素外边框 / 阴影
classes()	classes(add/remove/toggle/replace)	动态添加、移除、切换或替换 CSS 类 (如 no-shadow、border)
props()	props(add/remove)	添加或移除 Quasar 组件属性 (如 flat、bordered)
style()	style(add/remove/replace)	添加、移除或替换内联 CSS 样式 (如 padding: 0; margin: 10px;)
default_classes	default_classes(add/remove/toggle/replace) -> type[Self]	为该类所有元素设置默认 CSS 类 (需在实例化前调用)
default_props	default_props(add/remove) -> type[Self]	为该类所有元素设置默认属性 (需在实例化前调用)

方法名	语法	说明
<code>default_style</code>	<code>default_style(add/remove/replace) -> type[Self]</code>	为该类所有元素设置默认内联样式（需在实例化前调用）

2. 内容管理相关

方法名	语法	说明		
<code>clear()</code>	<code>clear() -> None</code>	移除所有子元素		
<code>remove()</code>	<code>`remove(element: Element</code> <code>int) -> None`</code>	移除指定子元素 (支持元素实例或 ID)		
<code>add_slot()</code>	<code>`add_slot(name: str,</code> <code>template: str</code>	<code>None = None) -> Slot`</code>	为卡片添加插槽 (基于 Vue 插槽机制, 适用于复杂布局)	
<code>move()</code>	<code>`move(target_container: Element</code> <code>None = None,</code> <code>target_index: int = -1,</code> <code>target_slot: str</code>	<code>None = None,</code> <code>target_index: int = -1,</code> <code>target_slot: str</code>	<code>None = None) -> None`</code>	将卡片移动到其他容器或插槽

3. 交互与绑定相关

方法名	语法	说明
<code>on()</code>	<code>on(type: str, handler: Callable, ...) -> Self</code>	绑定事件处理器（支持点击、鼠标按下等事件, 可指定 Python/JS 处理器）
<code>bind_visibility</code>	<code>bind_visibility(target_object: Any,</code> <code>target_name: str = 'visible', ...) -> Self</code>	双向绑定元素可见性到目标对象的属性
<code>bind_visibility_from</code>	<code>bind_visibility_from(target_object: Any,</code> <code>...) -> Self</code>	单向绑定可见性（从目标对象到卡片）
<code>bind_visibility_to</code>	<code>bind_visibility_to(target_object: Any, ...)</code> <code>-> Self</code>	单向绑定可见性（从卡片到目标对象）
<code>set_visibility</code>	<code>set_visibility(visible: bool) -> None</code>	直接设置元素可见性
<code>tooltip()</code>	<code>tooltip(text: str) -> Self</code>	为卡片添加悬停提示文本

4. 资源与生命周期相关

方法名	语法	说明	
<code>add_resource()</code>	<code>`add_resource(path: str</code>	<code>Path) -> None`</code>	为卡片添加资源（如 CSS、JS 文件目录）
<code>add_dynamic_resource()</code>	<code>add_dynamic_resource(name: str, function: Callable) -> None</code>	添加动态资源（通过函数返回资源响应）	
<code>delete()</code>	<code>delete() -> None</code>	删除卡片及所有子元素	
<code>update()</code>	<code>update() -> None</code>	同步更新元素到客户端界面	
<code>run_method()</code>	<code>run_method(name: str, *args: Any, timeout: float = 1) -> AwaitableResponse</code>	调用客户端方法（支持异步等待返回结果）	
<code>get_computed_prop()</code>	<code>get_computed_prop(prop_name: str, timeout: float = 1) -> AwaitableResponse</code>	获取客户端计算属性（需异步等待）	

5. 其他辅助方法

方法名	语法	说明
<code>mark()</code>	<code>mark(*markers: str) -> Self</code>	为元素添加标记（用于测试查询或依赖管理）
<code>ancestors()</code>	<code>ancestors(include_self: bool = False) -> Iterator[Element]</code>	迭代元素的祖先节点（可选包含自身）
<code>descendants()</code>	<code>descendants(include_self: bool = False) -> Iterator[Element]</code>	迭代元素的后代节点（可选包含自身）

五、使用场景与最佳实践

- 1. **内容分组展示**：将相关元素（如图片 + 描述文本、表单字段组、数据表格 + 操作按钮）封装在卡片中，提升界面可读性。
- 2. **模块化布局**：结合 `ui.row()`、`ui.column()` 等布局组件，用卡片实现网格化、分区化界面（如仪表盘、数据看板）。

3. **动态内容加载**: 通过 `clear()`、`remove()` 或数据绑定, 实现卡片内容的动态更新 (如根据用户操作切换卡片内数据)。
4. **样式统一化**: 使用 `default_classes()`、`default_props()` 为项目中所有卡片设置统一样式 (如统一边框、阴影强度), 保证界面一致性。

六、注意事项

1. 版本兼容性: `html_id` 属性需 2.16.0+ 版本, `toggle` 参数在 `default_classes()` 中需 2.7.0+ 版本, 使用时需确认 NiceGUI 版本。
2. `tight()` 方法: 启用后会移除内边距并隐藏嵌套元素的外边框 / 阴影, 适用于需要紧凑布局的场景 (如卡片内嵌套表格、列表)。
3. 事件绑定: `on()` 方法的 `throttle`、`leading_events`、`trailing_events` 参数仅对服务端事件处理器有效, 客户端 JS 处理器不受影响。
4. 插槽使用: 复杂布局需使用 `add_slot()` 时, 需了解 Vue 插槽机制, 避免嵌套插槽使用混乱。